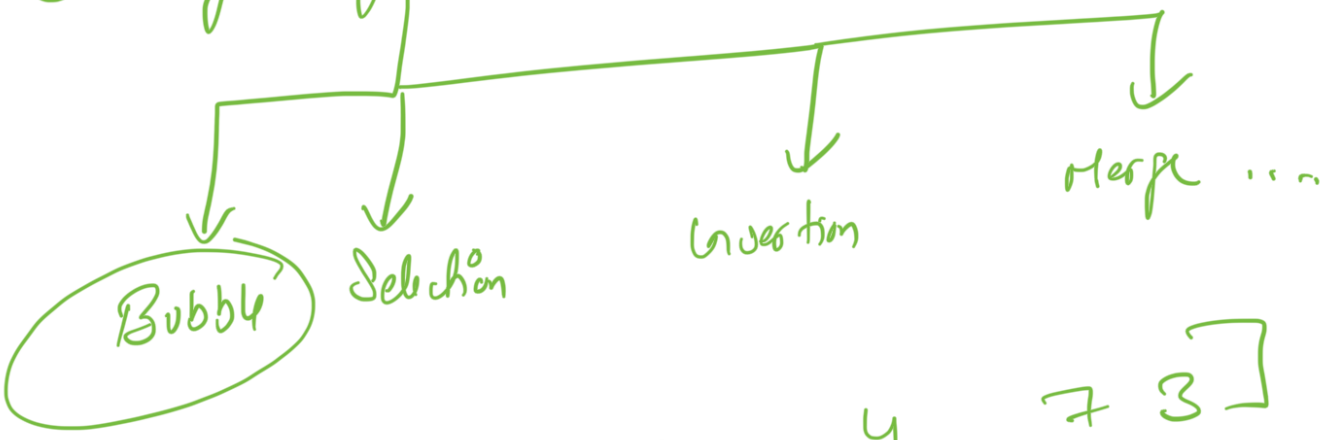
OOPS - 1 Notes

Bubble Sort

- Object Oriented Programming (OOP)
  - 4 pillars of OOP
- Constructor
  - Life cycle of an object



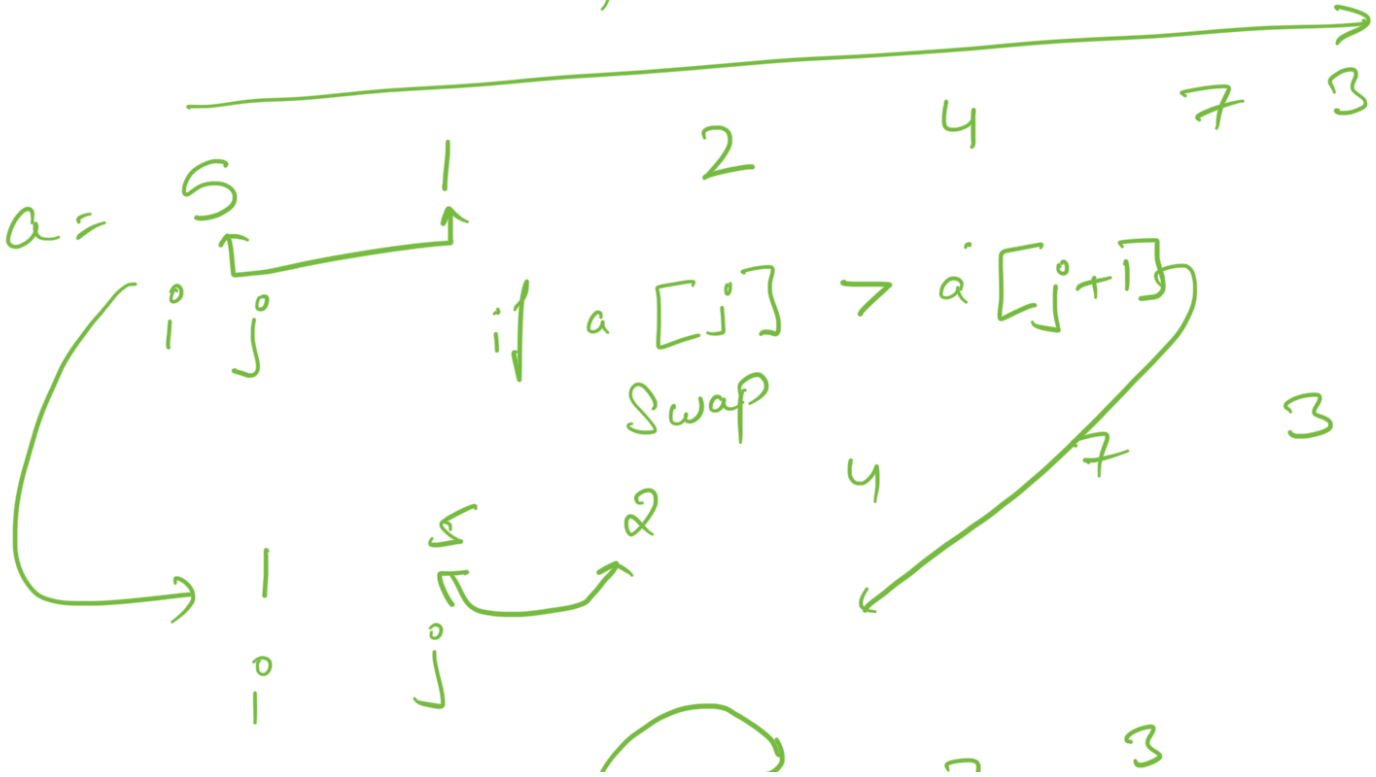
Sorting Algorithms

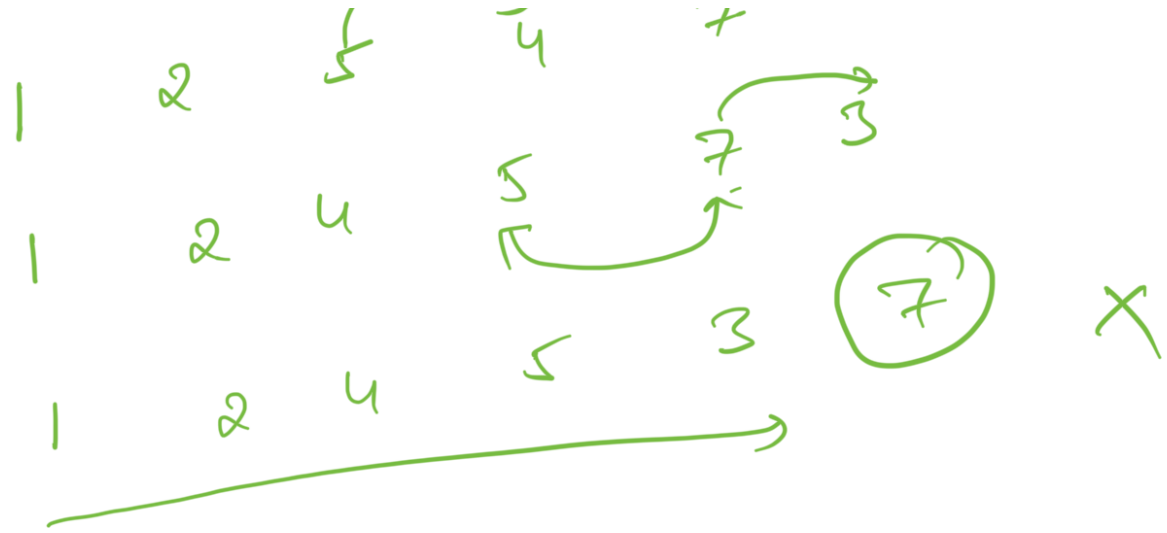




Algo:-

1. Repeatedly Compares adjacent elements
2. if they are in wrong order Swap them.
3. Largest elements bubble up to the end with each pass.





Time Complexity :-

OOP :- object oriented Programming ↳ it is a programing Paradigm ↳ concept of classes & objects

↳ class = it is a blueprint that defines the methods & properties of class

Analogy of class

↳ Think of a house blueprint made by an architect

↳ blueprint defines how house will look (wall, door, ...)

↳ in python, a class defines the structure (attributes and methods)

↳ variables ↳ functions

↑ object → Real world instance built from

(2) Object

↳ a real house blueprint is an object

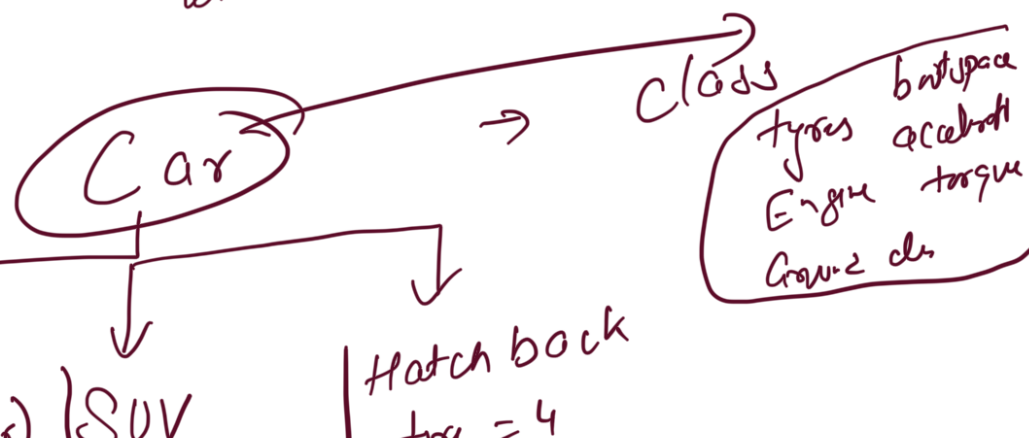
↳ you can build many houses (objects) from same blueprint (class)

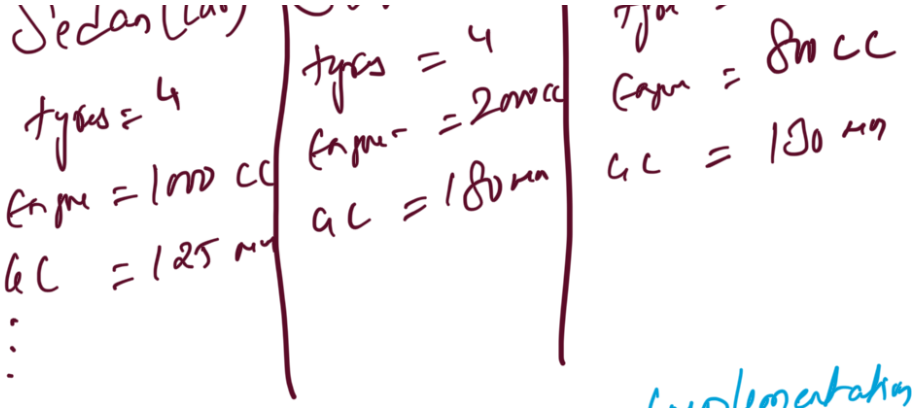
↳ Each house may have different paint colors, furniture or owners but they follow the same design.

↳ pillar of OOP

(a) Inheritance
(b) Abstraction
(c) Polymorphism
(d) Encapsulation

(1) Inheritance :- object of one class can inherit the features of other class.





(2) Abstraction :- Hiding implementation details, showing only essential features

↳ It focuses on what an object does, not how it does it.

↳ In python, we do implement Abstraction using Abstract Base classes (ABC)

(3) Polymorphism :- Same entity but different behavior.

print(1 + 2) = 3 → Int

print("1 + 2") = 1 + 2 → String

(4) Encapsulation :- Wrapping data (attributes) and methods (functions) into a

(4) Encapsulation



Single unit (class)

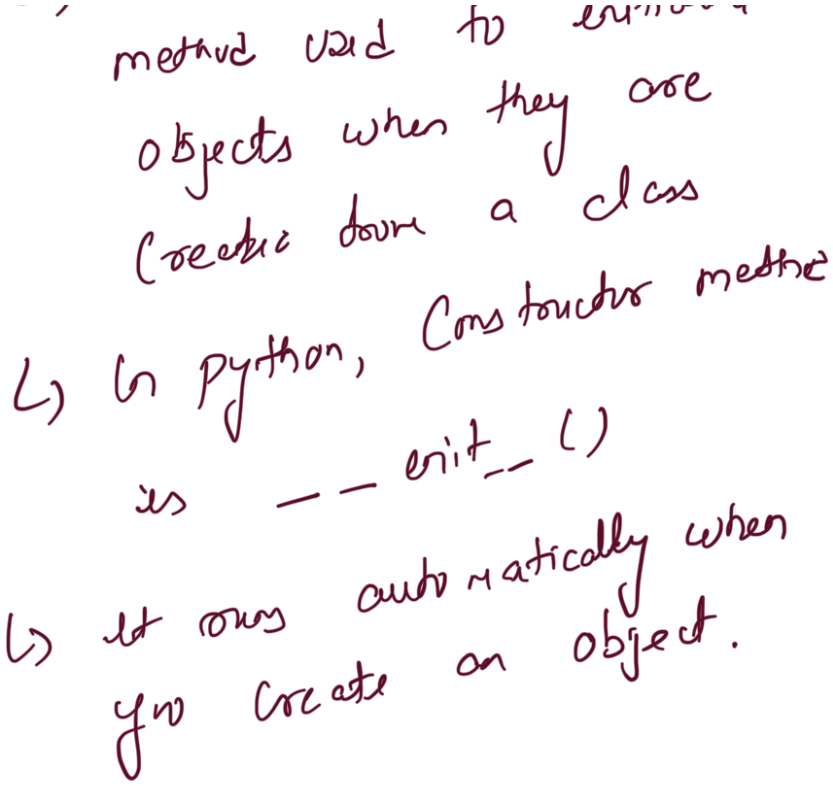
→ It also means restricting direct access to data by making variables as private and providing controlled access via methods.

↳ Data hiding and security.

Abstraction	Encapsulation
→ Hiding implementation details, showing only essential features	→ Hide data inside a class, restricting direct access
→ Focus on what an object does	→ Focus on how data is protected
→ We implement via ABC	→ We implement with private variables (__var) and getter/setter methods.

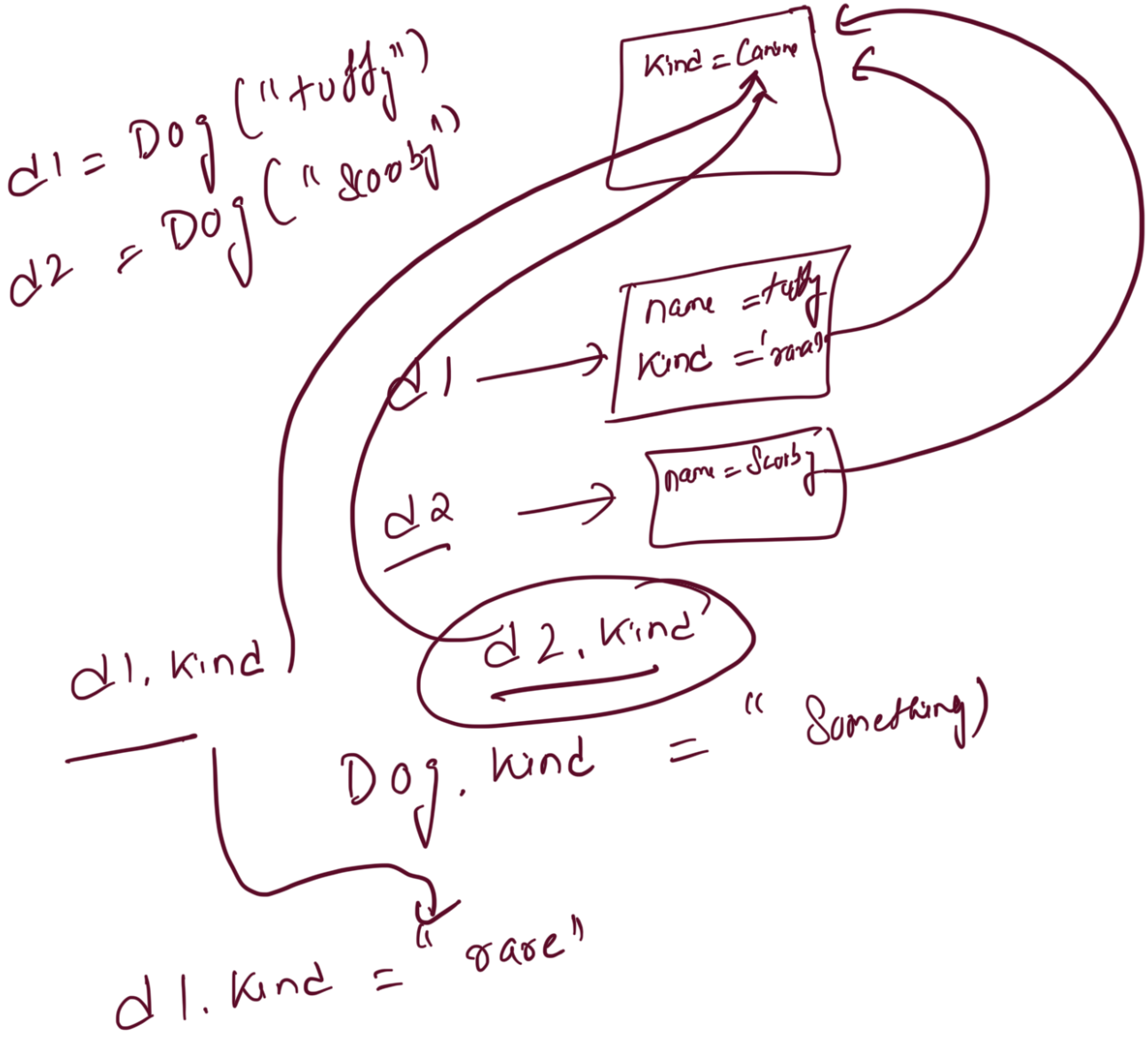
Constructor

↳ A Constructor is a special




```
class Dog:
  kind = "Canine"

  def __init__(self,name):
    self.name = name
```



class variables	Instance variables
① Shared by all objects of class	① Belong to a specific object.
② Define inside class but outside methods	② Define inside __init__ or methods using self
③ Stored once in memory,	③ Each object gets its own copy
④ changes affect all objects	④ change affects only [illegible] object